DATA STRUCTURES & ALGORITHMS

Previous Year Questions — Complete Solutions

Module 1 · Module 2 · Module 3

MODULE 1

Q1. Define Data Structure. Explain Types of Data Structures.

Definition

A data structure is a systematic and organised way of storing, organising, and managing data in a computer's memory so that it can be accessed, processed, and modified efficiently. It defines the data stored, the relationships between elements, and the set of operations applicable.

Types

1. Primitive: Basic types directly supported by hardware — int, float, char, bool, pointer.

2. Non-Primitive: Derived types, further divided into:

A. Linear — elements in sequential order (one predecessor, one successor):

- Array — contiguous memory, fixed size, O(1) random access by index.
- Linked List — dynamic size, nodes linked by pointers, non-contiguous.
- Stack — LIFO; insertions and deletions only at the TOP.
- Queue — FIFO; insert at REAR, delete from FRONT.

B. Non-Linear — elements with hierarchical or network relationships:

- Tree — hierarchical; root with child subtrees. Applications: file systems, expression trees.
- Graph — G = (V, E); vertices connected by edges; directed or undirected. Applications: GPS, social networks.

Category	Structure	Key Property	Example Use
Primitive	int, float, char, bool	Hardware support	Variable storage
Linear	Array / Stack / Queue / LL	Sequential order	Undo, scheduling
Non-Linear	Tree / Graph	Hierarchical/Network	File system, GPS

Q2. Explain the Concepts of ADT with Examples.

Definition

An Abstract Data Type (ADT) is a mathematical model for a data type defined by its behaviour — the set of values and the operations that can be performed — rather than by its implementation. 'Abstract' means the implementation is hidden from the user.

Key principle: ADT specifies WHAT operations do, not HOW they are implemented.

Characteristics

- Abstraction — user sees only the interface, not internal details.
- Encapsulation — data and operations bundled together.
- Reusability — same ADT usable across different programs.
- Modularity — implementation can change without affecting users.

Example 1 — Stack ADT

Data: Collection of elements with a TOP pointer (TOP = -1 means empty).

- push(x) — insert at top; pop() — remove from top; peek() — view top
- isEmpty(), isFull() — check stack status

Constraint: LIFO. The ADT does not specify array or linked list implementation.

Example 2 — Queue ADT

Data: Collection with FRONT and REAR pointers.

- enqueue(x) — insert at REAR; dequeue() — remove from FRONT

Constraint: FIFO.

Example 3 — List ADT

Data: Ordered sequence (a_0, a_1, ..., a_{n-1}).

- insert(pos,x), delete(pos), get(pos), search(x), size(), isEmpty()

ADT (Abstract)	Data Structure (Concrete)
Defines WHAT operations exist	Defines HOW operations are implemented
Mathematical / logical concept	Physical memory implementation
Multiple implementations possible	One specific implementation chosen

---

Q3. Define Algorithm and List Its Properties.

Definition

An algorithm is a finite, well-defined sequence of instructions that takes zero or more inputs and produces one or more outputs, solving a given problem. It must terminate in a finite number of steps.

Properties of an Algorithm (Knuth's 5)

Property	Meaning	Violation Example
Finiteness	Must terminate after a finite number of steps	Infinite loop: while(true){...}
Definiteness	Every step must be precisely and unambiguously defined	'Do something with x' — vague
Input	Zero or more well-defined inputs	Using an undefined variable
Output	One or more meaningful outputs related	No return value

Property	Meaning	Violation Example
	to inputs	
Effectiveness	Every step must be doable manually with basic tools	Unsolvable sub-step

Example — Find the Larger of Two Numbers

```
Step 1: Read A, B
Step 2: If A > B, print A; else print B
Step 3: STOP
```

Q4. Analyze Efficiency of Algorithms: Big O, Big Ω, and Big Θ.

Asymptotic notations describe the growth rate of an algorithm's time/space complexity as $n \to \infty$, independent of hardware.

A. Big O — O(g(n)) [Upper Bound / Worst Case]

Definition: $f(n) = O(g(n))$ if $\exists$ constants c, n_0 such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$

Meaning: f(n) grows NO FASTER than g(n). Gives the worst-case guarantee.

- $f(n) = 3n + 5 \to O(n)$. [Choose $c=4, n_0=5$: $3n+5 \le 4n$ ✓]
- Linear Search (element not found) → O(n); Bubble Sort → $O(n^2)$

B. Big Omega — Ω(g(n)) [Lower Bound / Best Case]

Definition: $f(n) = \Omega(g(n))$ if $\exists$ constants c, n_0 such that $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$

Meaning: f(n) grows NO SLOWER than g(n). Gives the best-case guarantee.

- $f(n) = 3n + 5 \to \Omega(n)$. [Choose $c=1, n_0=1$: $3n+5 \ge n$ ✓]
- Linear Search (element at index 0) → Ω(1)

C. Big Theta — Θ(g(n)) [Tight Bound / Exact]

Definition: $f(n) = \Theta(g(n))$ if $\exists$ constants c_1, c_2, n_0 such that $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$

Equivalently: f(n) = O(g(n)) AND f(n) = Ω(g(n)). Gives the exact growth rate.

- $f(n) = 3n + 5 \to \Theta(n)$. [$c_1=1, c_2=4, n_0=5$: $n \le 3n+5 \le 4n$ ✓]
- Merge Sort → Θ(n log n) [best AND worst case are both n log n]
- Quick Sort: $O(n^2)$ worst, Ω(n log n) best → NO single Θ for all cases

Notation	Bound	Case	Condition	Example
Big O	Upper	Worst case	$f(n) \le c \cdot g(n)$	Linear Search: O(n)
Big Ω	Lower	Best case	$f(n) \ge c \cdot g(n)$	Linear Search: Ω(1)
Big Θ	Tight	Average/Exact	$c_1 \cdot g \le f \le c_2 \cdot g$	Merge Sort: Θ(n log n)

Common Complexities (best → worst)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Q5. Explain Time and Space Complexity.

Time Complexity

Time complexity measures the number of basic operations (comparisons, assignments) an algorithm performs as a function of input size n — machine-independent.

Case	Notation	Definition	Example (Linear Search)
Best Case	Ω	Min operations on most favourable input	Element at index 0 → 1 comparison
Worst Case	O	Max operations on most unfavourable input	Element not present → n comparisons
Average Case	Θ	Expected operations over all inputs	Element at random → n/2 comparisons

Space Complexity

Space Complexity = Fixed Space + Variable Space

- Fixed Space — code, constants, simple variables (independent of n)
- Variable / Auxiliary Space — extra arrays, recursion stack (depends on n)

Algorithm	Time	Space (Aux)	Note
Bubble Sort	$O(n^2)$	O(1)	In-place, no extra array
Merge Sort	O(n log n)	O(n)	Needs temp array of size n
Binary Search (iter)	O(log n)	O(1)	Just two pointers
Binary Search (rec)	O(log n)	O(log n)	Recursion stack depth = log n

---

Q6. Differentiate Between the Following.

6a. Array vs Linked List

Basis	Array	Linked List
Memory	Contiguous (continuous)	Non-contiguous (scattered)
Size	Fixed (static)	Dynamic
Access time	O(1) — direct index	O(n) — sequential traversal
Insertion/Deletion	O(n) — requires shifting	O(1) — change links only
Memory usage	Less (no pointers)	More (pointer per node)
Cache friendliness	High	Low

6b. Stack vs Queue

Parameter	Stack	Queue
Principle	LIFO — Last In First Out	FIFO — First In First Out
Insertion	push() at the TOP	enqueue() at the REAR
Deletion	pop() from the TOP	dequeue() from the FRONT
Pointers	One — TOP	Two — FRONT and REAR

Parameter	Stack	Queue
Analogy	Stack of plates	People in a queue
Use cases	Recursion, undo, expression eval	Scheduling, BFS, print spooling

6c. Top-Down vs Bottom-Up Approach

Basis	Top-Down	Bottom-Up
Direction	Main problem → sub-problems	Sub-problems → main problem
Design method	Stepwise refinement	Composition of modules
Focus	Overall structure first	Detailed components first
Used in	Recursive solutions	Dynamic programming
Testing	Requires stubs	Requires drivers

6d. Big-O vs Big-Omega vs Big-Theta

Aspect	Big-O (O)	Big-Omega (Ω)	Big-Theta (Θ)
Bound type	Upper bound	Lower bound	Tight bound
Case	Worst case	Best case	Average / exact
View	Maximum time	Minimum time	Exact growth rate
Condition	$f(n) \leq c \cdot g(n)$	$f(n) \geq c \cdot g(n)$	$c_1 \cdot g \leq f \leq c_2 \cdot g$

MODULE 2

Q1. Define List Abstract Data Type (ADT).

A List ADT is a finite, ordered sequence L = (a_0, a_1, ..., a_{n-1}) of n elements where each element is identified by its position.

- n is the length; n = 0 means empty list.
- Order matters — position of each element is significant.
- Duplicates allowed. Dynamic — elements can be added or removed.
- Defined by its behaviour (operations), not its implementation.

Q2. List the Basic Operations Performed on List ADT.

Operation	Description	Time (Array)	Time (Linked List)
insert(pos, x)	Insert element x at position pos	O(n) — shifting	O(n) — traverse
delete(pos)	Remove element at position pos	O(n) — shifting	O(n) — traverse
get(pos)	Retrieve element at position pos	O(1) — direct index	O(n) — sequential
search(x)	Find position of element x	O(n)	O(n)
size()	Return number of elements	O(1)	O(1)
isEmpty()	Return true if list is empty	O(1)	O(1)
display()	Print all elements in order	O(n)	O(n)

Q3. Describe Array-Based Implementation of List with a Neat Diagram.

Concept

A List is stored in a contiguous array. An integer size tracks the current number of valid elements. MAX_SIZE is fixed at declaration.

```
struct List { int data[MAX_SIZE];  int size; };
```

Memory Layout Diagram (L = 10, 20, 30, 40, 50; MAX_SIZE = 8)

10	20	30	40	50	—	—	—
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

size = 5 (indices 0–4 valid; indices 5–7 are free slots)

Key Operations

```
// Insert at pos – shift right, place value
```

```
for(i=size; i>pos; i--)  data[i]=data[i-1];
data[pos]=value;  size++;

// Delete at pos — shift left
for(i=pos; i<size-1; i++)  data[i]=data[i+1];
size--;

// Get at pos — O(1) direct access
return data[pos];
```

Advantages	Disadvantages
O(1) random access by index	Fixed size — MAX_SIZE set at compile time
Cache-friendly, simple to implement	Insert/delete take O(n) due to element shifting

MODULE 3

Q1. Define Stack and List Its Basic Operations.

Definition

A Stack is a linear data structure following the LIFO (Last In First Out) principle — the element inserted last is removed first. All insertions and deletions happen at one end called the TOP. TOP = -1 means the stack is empty.

Operation	Description	Condition	Time
push(x)	Insert element x at the top	Stack must NOT be full	O(1)
pop()	Remove and return the top element	Stack must NOT be empty	O(1)
peek()	Return top element without removing	Stack must NOT be empty	O(1)
isEmpty()	Returns true if TOP == -1	—	O(1)
isFull()	Returns true if TOP == MAX-1	—	O(1)
size()	Returns TOP + 1	—	O(1)

Q2. Define Queue. Explain Different Types of Queue.

Definition

A Queue is a linear data structure following the FIFO (First In First Out) principle — the element inserted first is removed first. Insertions happen at the REAR and deletions at the FRONT. Two pointers FRONT and REAR track both ends.

Types of Queue

1. Linear Queue: Standard FIFO. REAR increments on enqueue; FRONT on dequeue.
Problem: Once FRONT advances, freed slots at beginning cannot be reused — wasted space.

2. Circular Queue: Solves the wasted-space problem by wrapping REAR using modulo arithmetic.

- REAR = (REAR + 1) % MAX_SIZE — wraps back to index 0 when end is reached
- Full condition: (REAR + 1) % MAX == FRONT

3. Priority Queue: Each element has a priority. Element with highest (or lowest) priority dequeued first, regardless of insertion order.

- Ascending (Min) PQ — smallest priority number served first
- Descending (Max) PQ — largest priority number served first
- Applications: OS scheduling, Dijkstra's algorithm, hospital triage

4. Deque (Double-Ended Queue): Insertions and deletions at BOTH ends.

- Input Restricted — insert only at REAR; delete from both ends
- Output Restricted — delete only from FRONT; insert at both ends

Type	Insertion	Deletion	Key Feature
Linear Queue	REAR only	FRONT only	Basic FIFO; wasted space issue
Circular Queue	REAR (wraps)	FRONT (wraps)	No wasted space; modulo wrap
Priority Queue	By priority	Highest priority first	Not strictly FIFO
Deque	Both ends	Both ends	Most flexible

Q3. List Applications of Stack.

- Reversing data — LIFO property naturally reverses any sequence (string, number, array).
- Infix to Postfix/Prefix conversion — operator precedence managed via stack.
- Postfix expression evaluation — push operands; on operator, pop two, apply, push result.
- Function call management (Call Stack) — stores return address, locals, parameters per call; popped on return.
- Undo / Redo — each change pushed; Ctrl+Z pops the last operation.
- Backtracking — maze solving, N-Queens, DFS; stack stores path; backtrack by popping.
- Syntax parsing — compiler checks balanced parentheses, brackets, braces.
- Browser history — Back button pops the previously visited page.

Q4. Apply ADT Concept to Explain Stack Operations (Push, Pop, Peek, isEmpty, isFull).

Stack ADT — Array Implementation in C

```
#define MAX 5
int stack[MAX], top = -1;

int isEmpty() { return (top == -1); }
int isFull()  { return (top == MAX-1); }

void push(int x) {
    if (isFull()) { printf("Overflow!\n"); return; }
    stack[++top] = x;        // increment top first, then store
}

int pop() {
    if (isEmpty()) { printf("Underflow!\n"); return -1; }
    return stack[top--];     // return value first, then decrement
}

int peek() {
    if (isEmpty()) return -1;
    return stack[top];       // no change to top
}
```

Operation Trace — push(10), push(20), push(30), pop(), peek()

Operation	Stack State	TOP	Returns
(initial)	[ ][ ][ ][ ][ ]	-1	—
push(10)	[10][ ][ ][ ][ ]	0	—
push(20)	[10][20][ ][ ][ ]	1	—
push(30)	[10][20][30][ ][ ]	2	—
pop()	[10][20][ ][ ][ ]	1	30
peek()	[10][20][ ][ ][ ]	1	20 (unchanged)

Q5. Describe Array Representation of Stack with a Neat Diagram.

Concept

A fixed-size array stores stack elements. Integer TOP points to the topmost element. TOP = -1 means empty. All push/pop/peek operations are O(1).

Diagram (MAX_SIZE = 6, after push 10, 20, 30, 40)

10	20	30	40	—	—
[0]	[1]	[2]	**[3] ← TOP**	[4]	[5]

TOP = 3 → top element = 40. Slots [4] and [5] are unused.

Operation	Condition	Code	Time
push(x)	TOP == MAX-1 → Overflow	stack[++top] = x	O(1)
pop()	TOP == -1 → Underflow	return stack[top--]	O(1)
peek()	TOP == -1 → Empty	return stack[top]	O(1)
isEmpty()	—	return (top == -1)	O(1)
isFull()	—	return (top == MAX-1)	O(1)

Q6. Explain Linked List Representation of Stack.

Concept

Each stack element is a node with a data field and a next pointer. The HEAD pointer acts as the TOP. No fixed size — dynamic. No overflow unless heap is exhausted.

Node Structure & Diagram

```
struct Node { int data;  struct Node *next; };
struct Node *top = NULL;  // empty stack
```

After push(10), push(20), push(30):

TOP → [30 | •] → [20 | •] → [10 | NULL]

push(x) — Insert at HEAD

```
void push(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;  n->next = top;  // new node → old top
    top = n;                      // top updated
}
```

pop() — Delete from HEAD

```
int pop() {
    if (!top) { printf("Underflow!\n"); return -1; }
    struct Node *t = top;  int val = t->data;
    top = top->next;  free(t);  return val;
}
```

Feature	Array Stack	Linked List Stack
Size	Fixed (MAX_SIZE)	Dynamic — no fixed limit
Overflow	TOP == MAX-1	Only if heap is exhausted
Extra memory	None per element	Pointer field per node
push / pop time	O(1)	O(1)

Q7. Apply Stack Operations to Reverse a Given String or Number.

Principle

The LIFO property naturally reverses any sequence. Push all characters, then pop — they come out in reverse order.

C Implementation

```
void reverseString(char str[]) {
    int n=strlen(str);  char stack[100];  int top=-1;
    for(int i=0; i<n; i++) stack[++top]=str[i];  // push all
    for(int i=0; i<n; i++) str[i]=stack[top--]; // pop all
}
```

Trace — Reverse "HELLO"

Phase	Operation	Stack	Result
PUSH	push H,E,L,L,O	[H E L L O] ← TOP=O	
POP	pop → O	[H E L L]	O
POP	pop → L	[H E L]	OL
POP	pop → L	[H E]	OLL
POP	pop → E	[H]	OLLE
POP	pop → H	[ ]	OLLEH ✓

Q8. Convert Infix Expressions to Prefix and Postfix.

Quick Reference

Notation	Operator Position	Example
Infix	Between operands (A op B)	A + B
Postfix	After operands	A B +
Prefix	Before operands	+ A B

Precedence (High → Low): ^ (right-assoc) > *, /, % > +, −

Infix → Postfix Algorithm

- Operand → output directly.
- '(' → push. ')' → pop until '(', discard brackets.
- Operator → pop equal/higher precedence operators, then push current.
- End → pop all remaining operators to output.

Infix → Prefix: Reverse expression (swap brackets) → apply postfix algorithm → reverse result.

All Conversions Summary

Infix Expression	Postfix	Prefix
(A-B)*(C+D)	AB-CD+*	*-AB+CD
(A+B)*C-D/E	AB+C*DE/-	-*+ABC/DE
(A+B)/(C+D)-(D*E)	AB+CD+/DE*-	-/+AB+CD*DE
A-(B/C+(D%E*F)/G)*H	ABC/DE%F*G/+H*-	-A*+/BC/*%DEFGH
(A+(B*C)	ABC*+	+A*BC
(A-B/C)*(A/K-L)	ABC/-AK/L-*	*-A/BC-/AKL
(A+B)*C	AB+C*	*+ABC
(f-g)*((a+b)*(c-d)/e)	fg-ab+cd-*e/*	*-fg/*+ab-cde

Detailed Token Trace — (A+B)*C-D/E (representative example)

Token	Action	Stack	Output
(	Push '('	(	
A	Operand	(	A
+	Push (top='(')	(+	A
B	Operand	(+	AB
)	Pop until '('		AB+
*	Push (stack empty)	*	AB+
C	Operand	*	AB+C
-	prec(-)<prec(*); pop *;	-	AB+C*

Token	Action	Stack	Output
	push -		
D	Operand	-	AB+C*D
/	prec(/)>prec(-); push /	-/	AB+C*D
E	Operand	-/	AB+C*DE
END	Pop /, then -		AB+C*DE/-